

Chapter 1 Introduction

- What operating system do
- Computer-system organization
- Computer-system architecture
- OS operations
- resource management
- security and protection
- Virtualization
- Distributed systems
- Kernel data structures
- Computing Environments
- Free and open-source operating system

What does the term Operating System means

- A program that manages the computer hardware
- Basis for Application Programs
- Intermediary between users and hardware

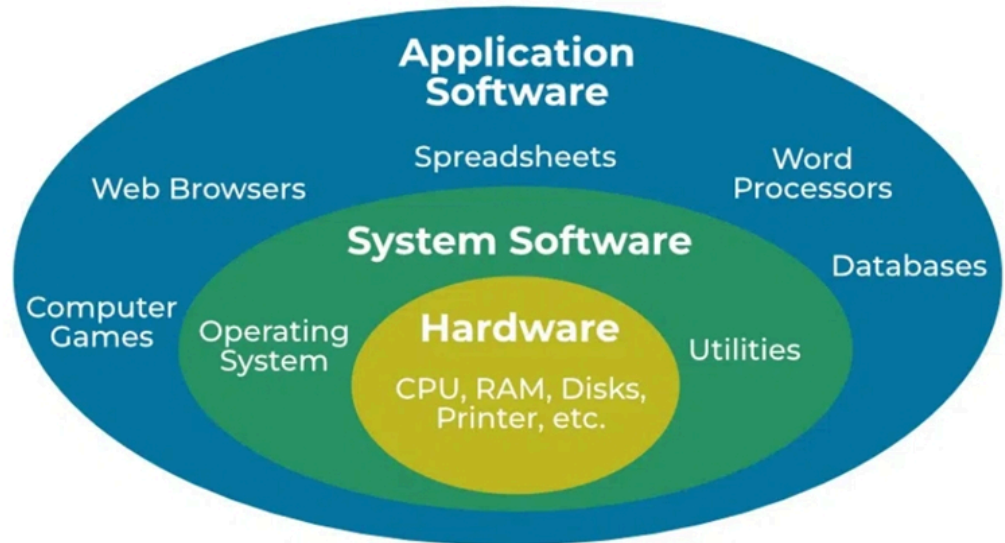
What is an Operating System

A program that acts as an intermediary between a user of a computer and the computer hardware.

Operating system goals:

- Execute user programs and make solving user problems easier.
- Make the computer system convenient to use

- Use the computer hardware in an **efficient** manner



System Software:

- Manages and operates the computer hardware.
- Acts as a platform on which application software can run.
- Runs in the background and ensures the system works efficiently.
- Examples: Operating systems (Windows, Linux, macOS), device drivers, and utility programs.

Application Software:

- Designed to help users perform specific tasks or activities.
- Depends on system software to function but focuses on user-oriented tasks.
- Typically runs in the foreground (i.e., you open the application to work with it).
- Examples: Microsoft Word, Adobe Photoshop, web browsers, and games.

Computer system Structure

- Hardware
- Operating system
- Application programs
- Users

The order is from bases to top. If no operating system, we need to **manually operate the computer hardware**.

What Operating Systems Do -- User View

- Convenience, **ease of use and good performance**

- Not for resource utilization.

With different shapes for shared computer, handheld computers etc.

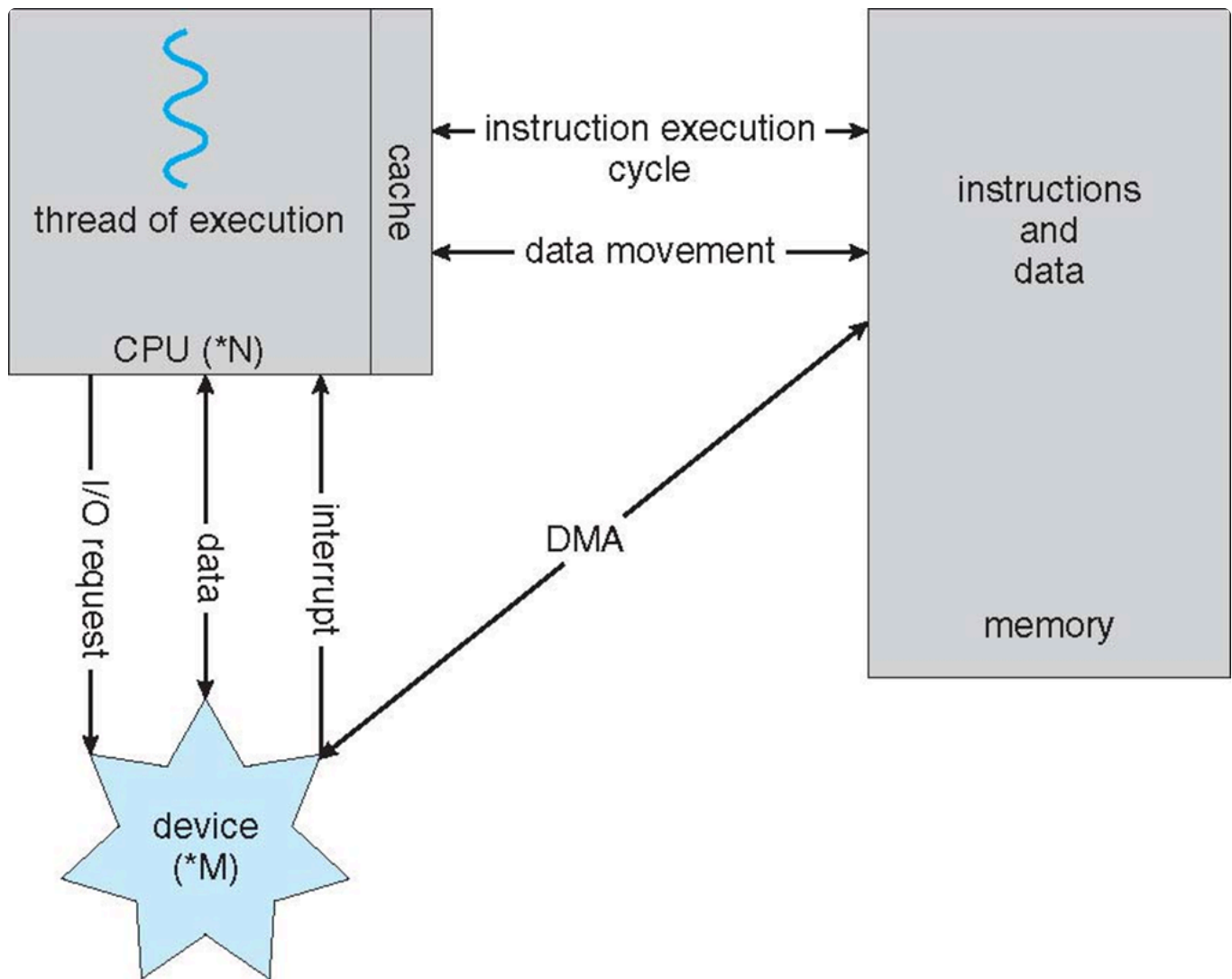
What Operating Systems Do -- System View

- A resource allocator. Manage all resources and conflicting requests for efficient and fair resource use.
- A control program. Prevent errors and improper use of the computer especially for I/O.

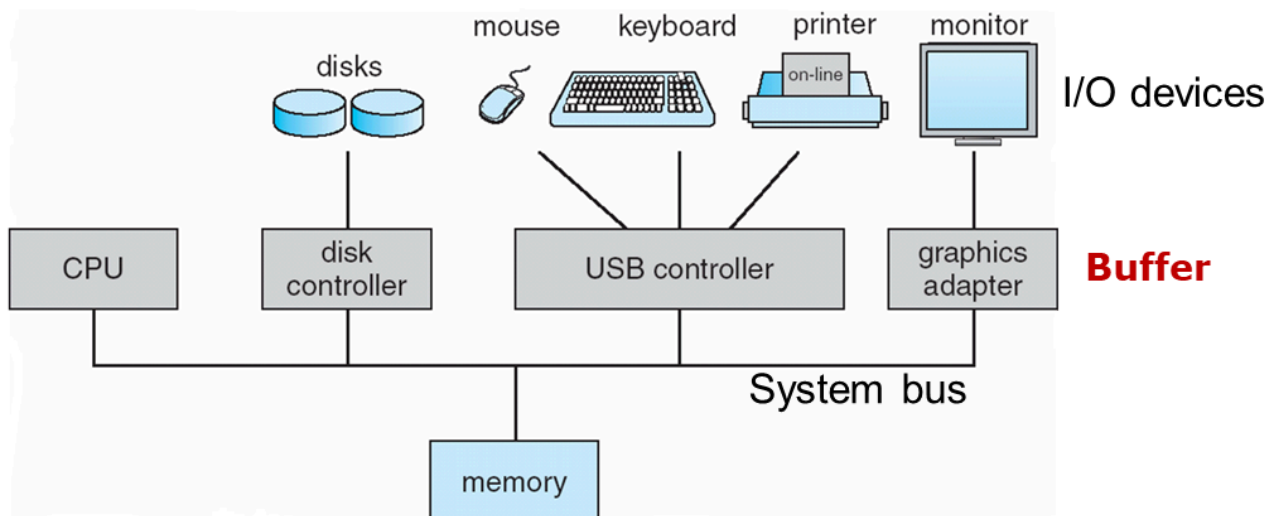
Computer Startup

- Bootstrap program is load at power-up or reboot.
 - Loads operating system kernel and starts execution
 - Locate the operating-system kernel and load it into memory
 - Some services are provided outside of the kernel by system software
 - Typically stored in ROM, known as firmware
- Kernel is the core components of an operating system.
 - manages fundamental tasks such as CPU scheduling, memory management and hardware access
 - Runs in a privileged mode(kernel space) with full access to the system's hardware.

How a Modern Computer Works



Computer System Organization



One or more CPUs, device controllers connect through common bus to share memory.

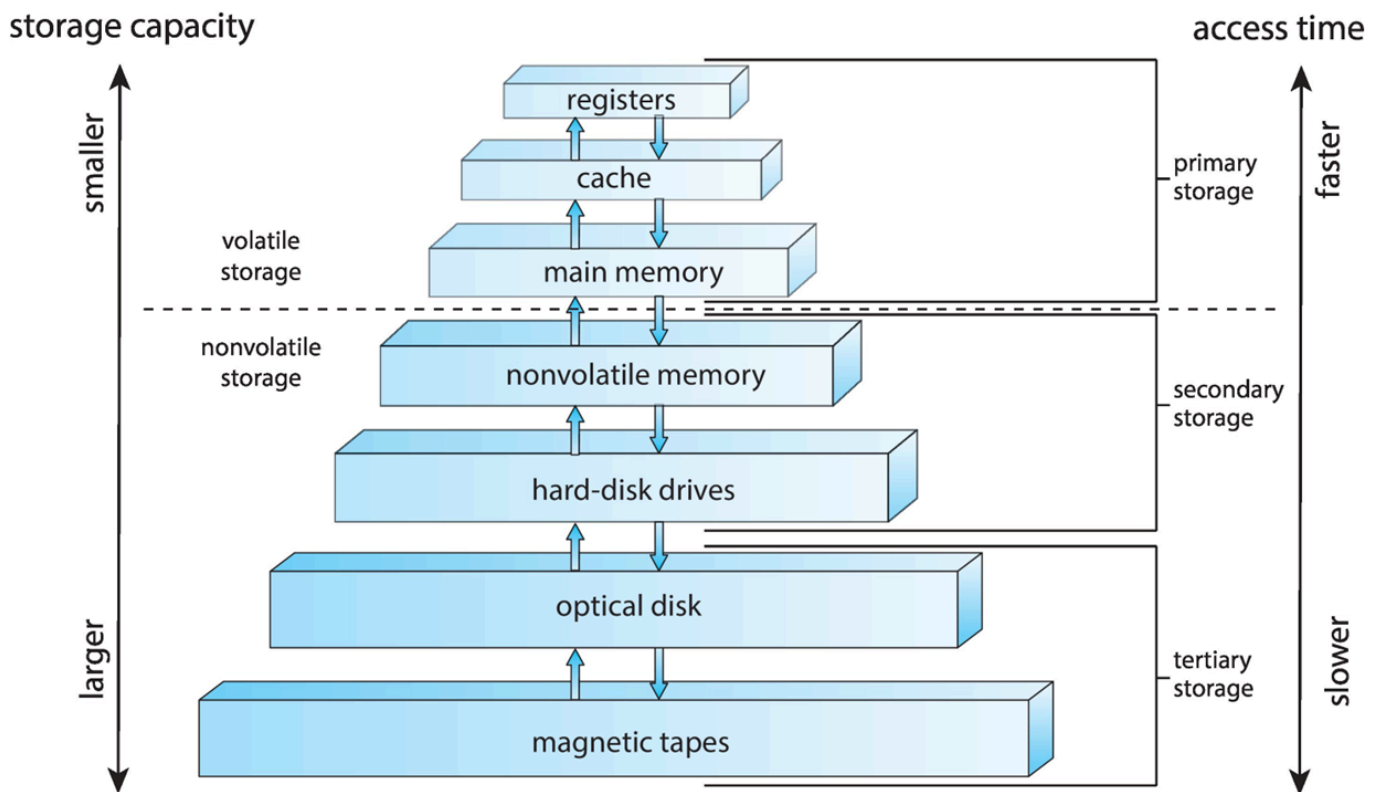
- Each controller is in charge of a particular device
- The CPU and the controllers can execute concurrently, competing for memory cycles.

- **Memory controller** is provided to ensure **orderly access** to shared memory, or, **synchronize access** to the memory.

Storage Structure

- Main memory-only large storage media that **CPU can access directly**
 - **Random access(RAM)**
 - Typically **volatile**
 - Typically random-access memory in the form of **dynamic random-access memory(DRAM)**
- Secondary storage -- extension for main memory with **large nonvolatile storage**.
- Hard Disk Drives(HDD) -- disk surface is divided into tracks which are subdivided into **sectors**. Disk controller determines the logical interaction between disks and the computer.
- Non-volatile memory(NVM) devices -- faster than hard disks and nonvolatile.

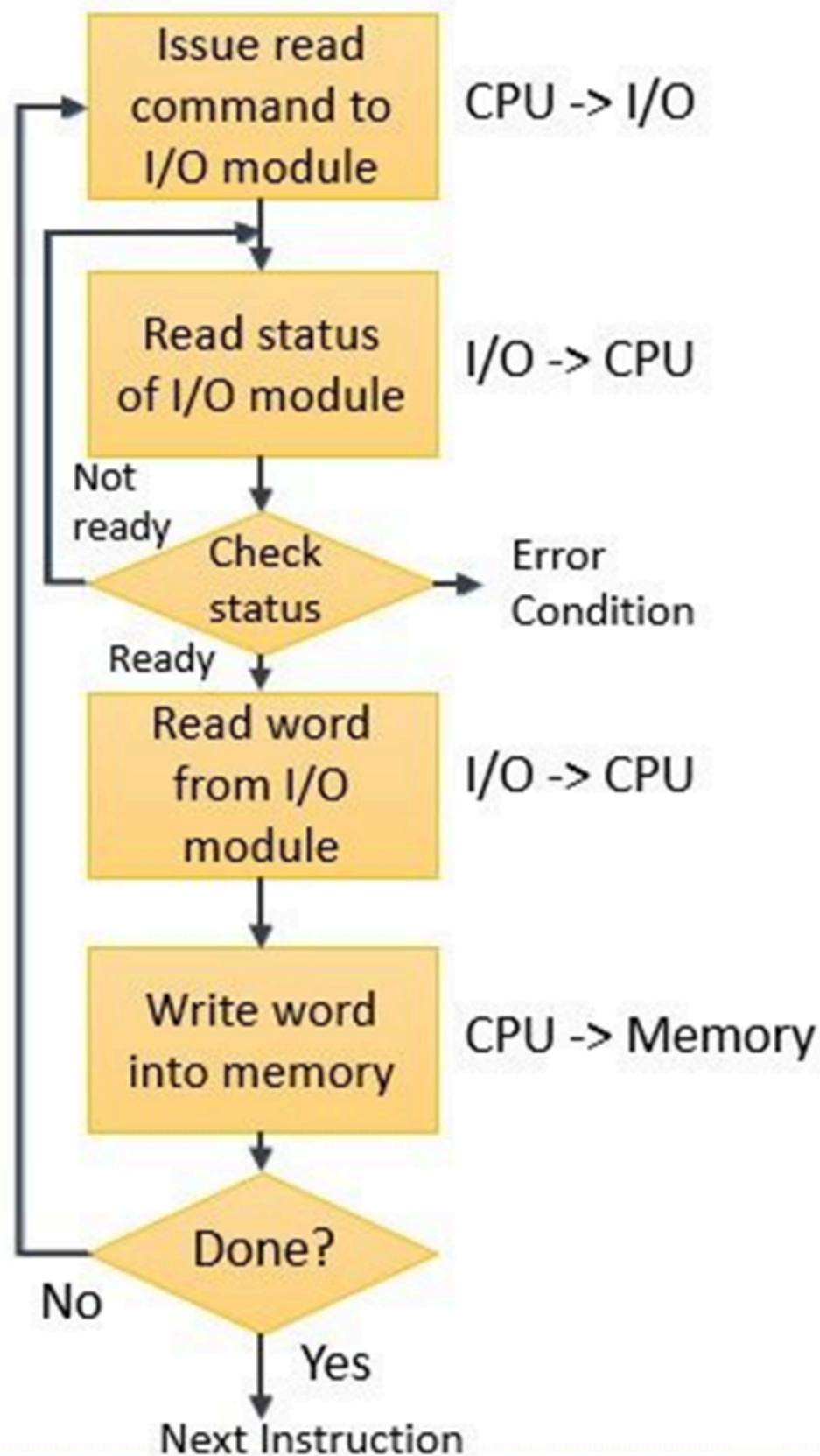
Storage Hierarchy



I/O Operations

Programmed I/O

A technique that we use to transfer data between the processor and the I/O module.



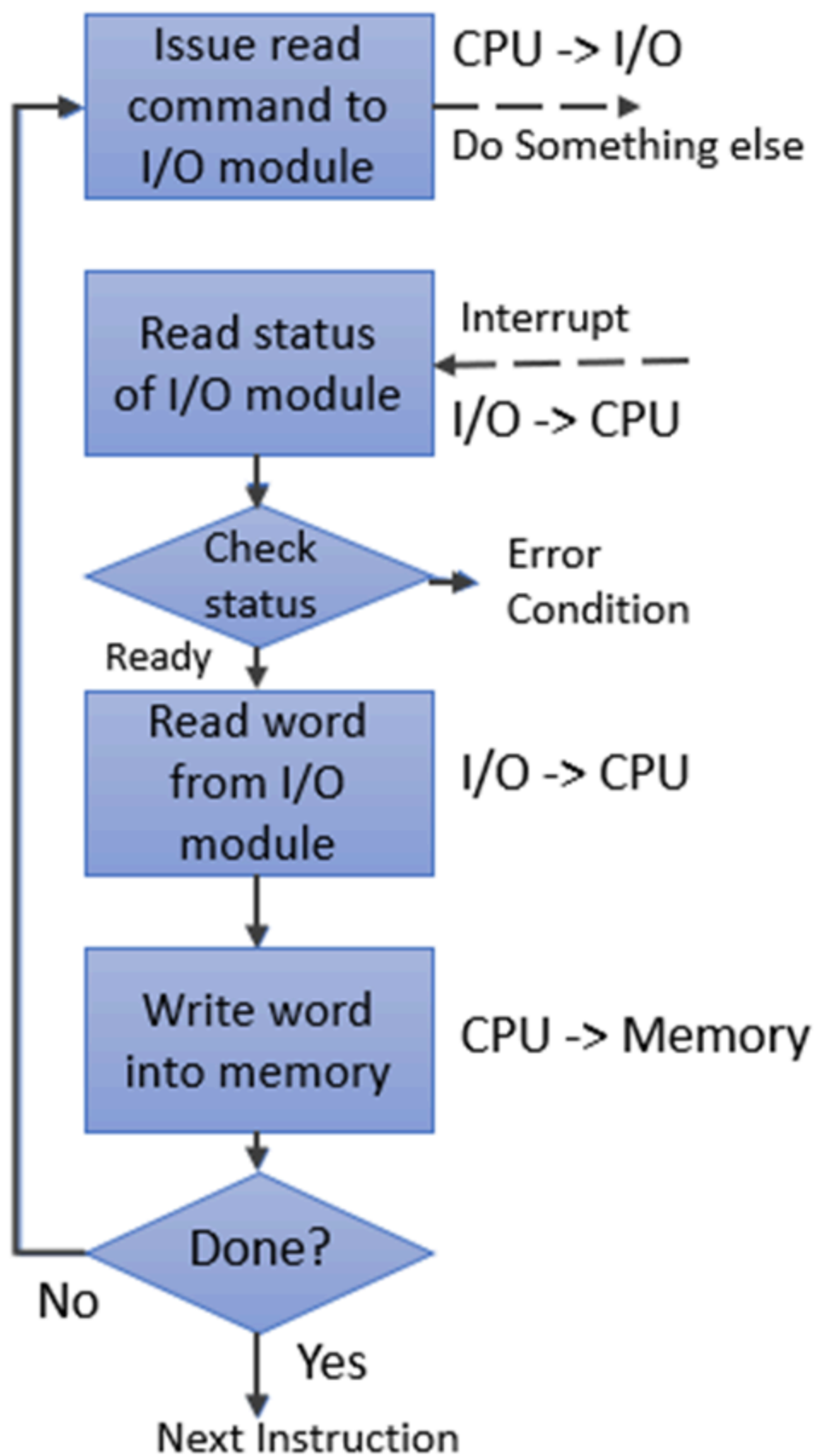
Programmed I/O to transfer data from I/O module to memory

The CPU periodically keeps on checking whether the I/O module is ready for handling data or whether it has completed the task until it is completed.

The long waiting of processor deteriorates the performance of the system.

Interrupted I/O

An approach to transfer data between memory and I/O devices through processor.



Interrupted I/O to Transfer Data
from I/O Module to Memory

It involves the use of **interrupt** to exchange data between I/O and memory.

After issuing the I/O command to I/O module, the processor **does some other work** until the I/O has been performed and issues an interrupt signal to the processor.

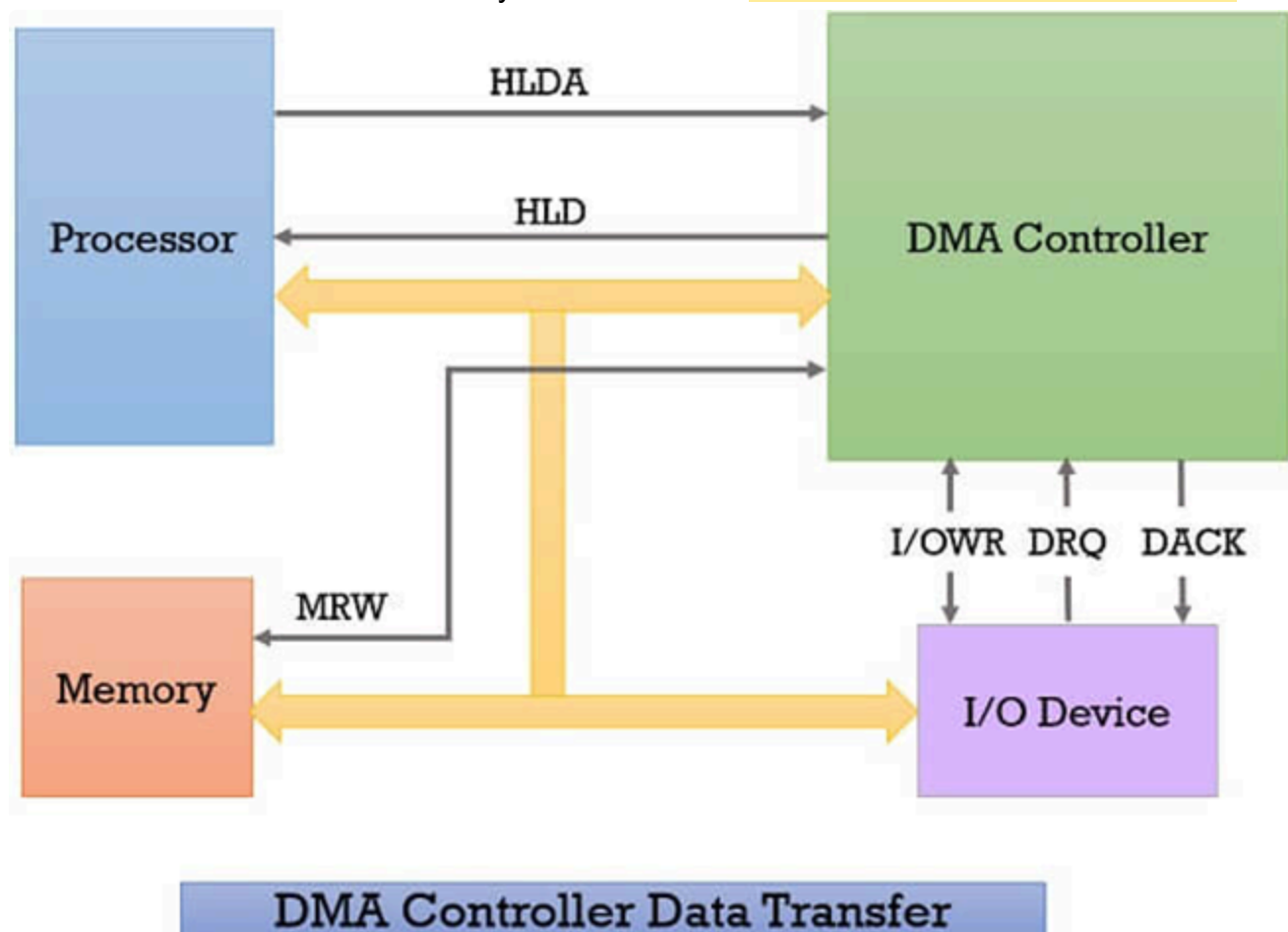
Programmed I/O V.S. Interrupted I/O

Programmed I/O is **easy to perform** but it will **waste the resource of processor** and **has a slow response speed**.

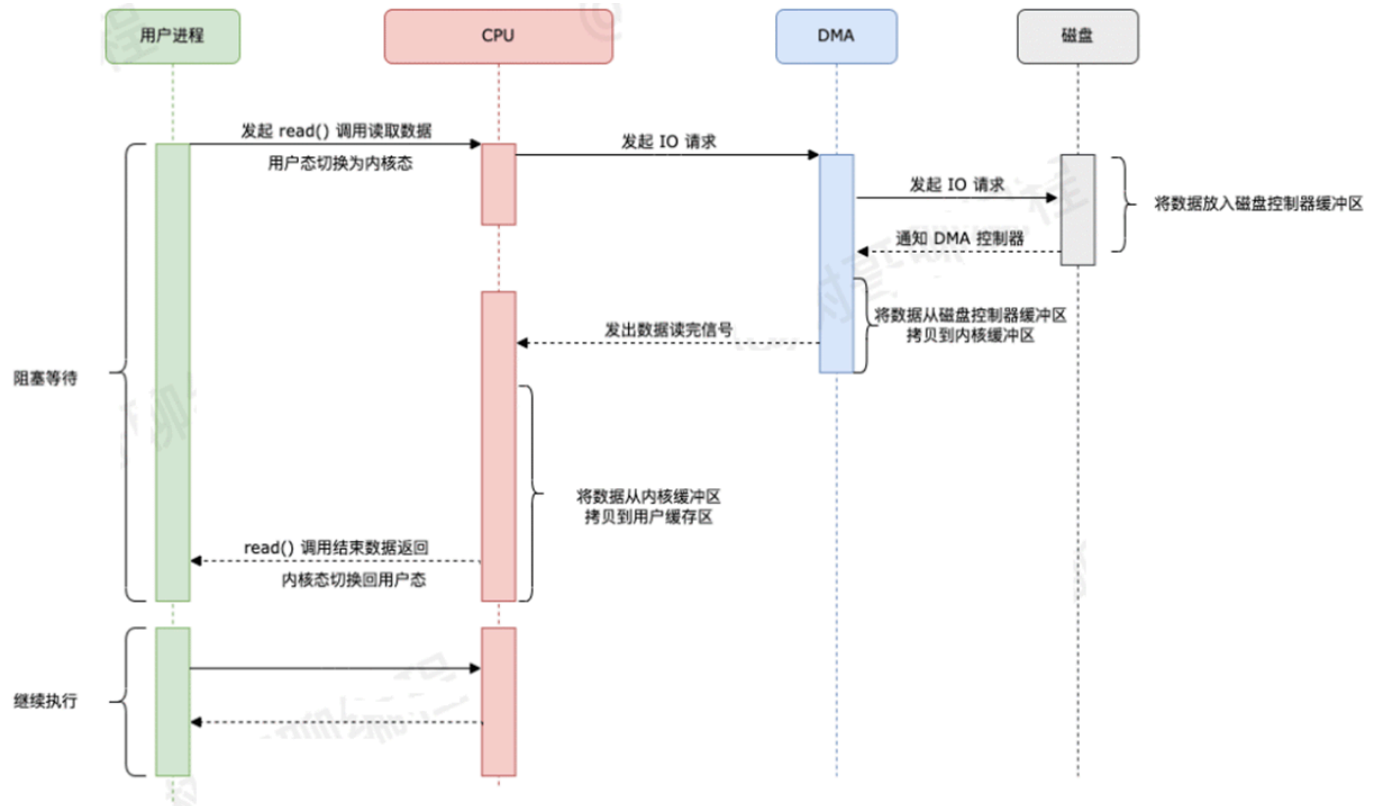
Interrupted I/O **decreases the number of polling operations**, **improving the response speed** and **and the utilization rate**. But it will **occupy the time of CPU** and **need the design and performance of interrupted program**.

Direct Memory Access(DMA)

Transfers the data between memory and I/O devices **without the intervention of the CPU**.



It reduces the participation of CPU.



When I/O device wants to transfer data to or from memory, it sends the DMA request(DRQ) to DMA controller. And it will asks the CPU to hold for a few clock cycles by sending it the Hold request(HLD).

CPU receives HLD and relinquishes the bus and sends the Hold acknowledgement(HLDA) to DMA controller.

After receiving HLDA, DMA controller acknowledges I/O device (DACK) to transfer the data and takes the charge of system bus and transfer the data to or from memory.

When the data transfer is accomplished, the DMA raise an interrupt to CPU to let it know the task is finished and the processor can take control over the bus again and start processing where it has left.

The process is like:

- I/O -> DMA controller
- DMA controller -> CPU -> DMA controller
- I/O <-> DMA controller <-> memory
- DMA controller -> CPU

For DMA,

- Device controller transfers blocks of data from buffer storage(controllers) directly to main memory
- Only one interrupt is generated per block rather than one interrupt per byte(for programmed

I/O or Interrupted I/O).

- Used for high-speed I/O devices to transmit information close to memory speeds.

Three modes for DMA controller to transfer data:

- Burst Mode:
Once DMA controller gains the charge of system bus, it releases the bus **only after completion of data transfer**. CPU has to wait for the system buses.(But can handle instructions in its cache without access memory)
- Cycle Stealing Mode:
DMA controller **forces the CPU to stop its operation** and **relinquish the control over bus for a short term to DMA controller**. DMA controller releases the bus **after transfer of every byte** and then **again requests for the system bus**. In this way, DMA controller steals the clock cycle for transferring every byte.
- Transparent Mode:
DMA controller takes charge of system bus **only if the processor does not require the system bus**.

Advantages:

- Without the involvement of CPU, **speed up the read-write operations**.
- **Reduce the clock cycle requires** to read or write a block of data.
- Reduces the overhead of processor.

Disadvantages:

- Cost to implement a DMA controller in the system as a hardware unit.

Why Only One Byte (or Word) per Interrupt for programmed I/O or Interrupted I/O

- Device controllers typically provides a **single data register** for device and if it is ready or full, the CPU must **interrupt(interrupted I/O)** or **poll(programmed I/O)** to move just that one byte before another arrives.
- DMA controller can **transfer larger blocks without CPU intervention** hence it is more effective.

Interrupts

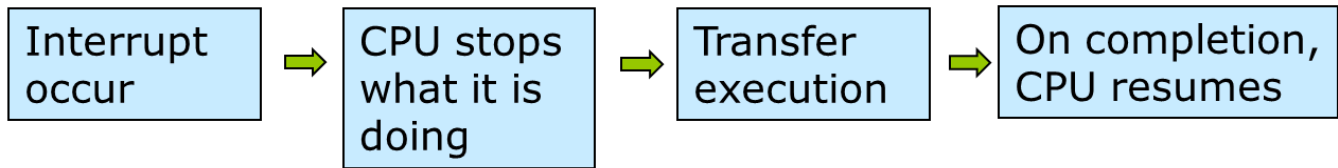
Events are almost always signaled **by the occurrence of an interrupt or a trap**.

Interrupt driven includes hardware and software

- Hardware Interrupt
Triggered by devices(such as I/O operations is completed)

- Software interrupt (**exception or trap**)
 - Software error(e.g. division by zero)
 - Invalid memory access
 - A specific request from user program that an OS service be performed

An **interrupt service routine** is provided to deal with the interrupt.



Interrupt transfers control to the **interrupt service routine** generally through **interrupt vector**, which is **a table of pointers** containing **the addresses of all the service routines**.

Every interrupt type is assigned a number and the vector determines what code is invoked to handle the interrupt.

0	Divide Error
2	Non-Maskable Interrupt
3	Breakpoint Exception
6	Invalid Opcode
11	Segment Not Present
12	Stack-Segment Fault
13	General Protection Fault
14	Page Fault
18	Machine Check
32-255	User Defined Interrupts

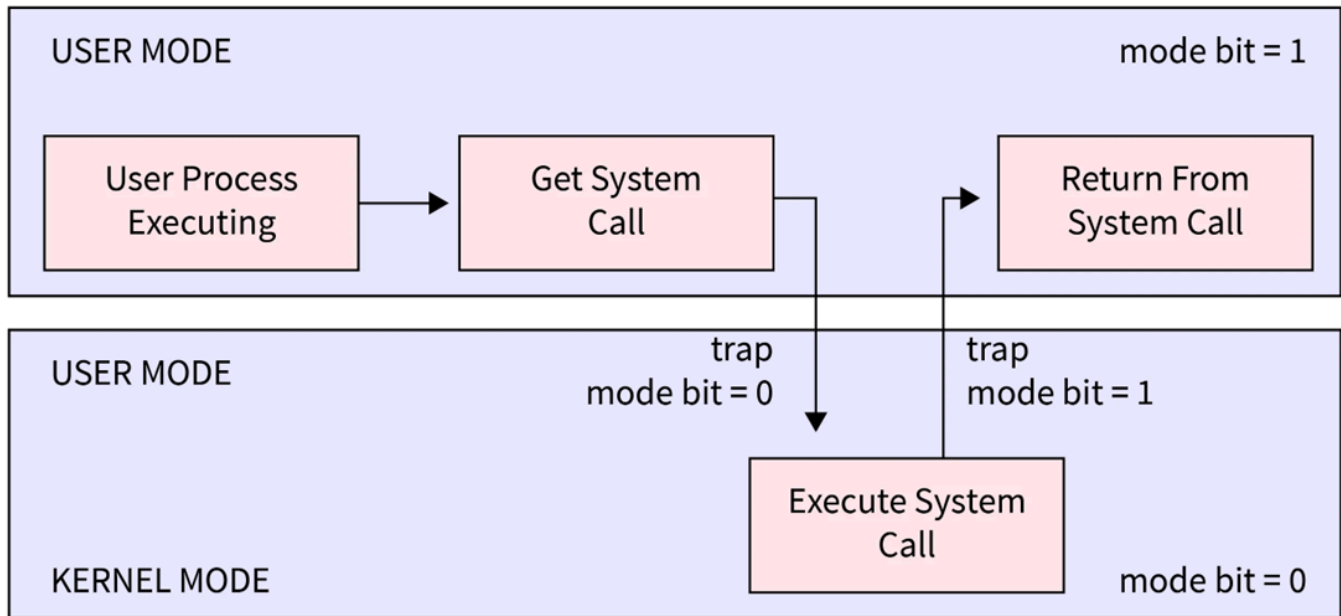
Thus, the **interrupt service routine** is called **indirectly** through the table to let interrupt transfer control to appropriate interrupt service routine.

User Mode V.S. Kernel Mode

For brief,

- User mode:
 - Operate within strict limits. Cannot directly operate hardware or system resources.
 - Safe, controlled and limited.
- Kernel mode:
 - Can manage low-level access to CPU, memory and devices.

Powerful but risky.



Why two modes:

- Security and Stability
Keep most software in user mode to avoid crash the entire system
- Controlled access to hardware

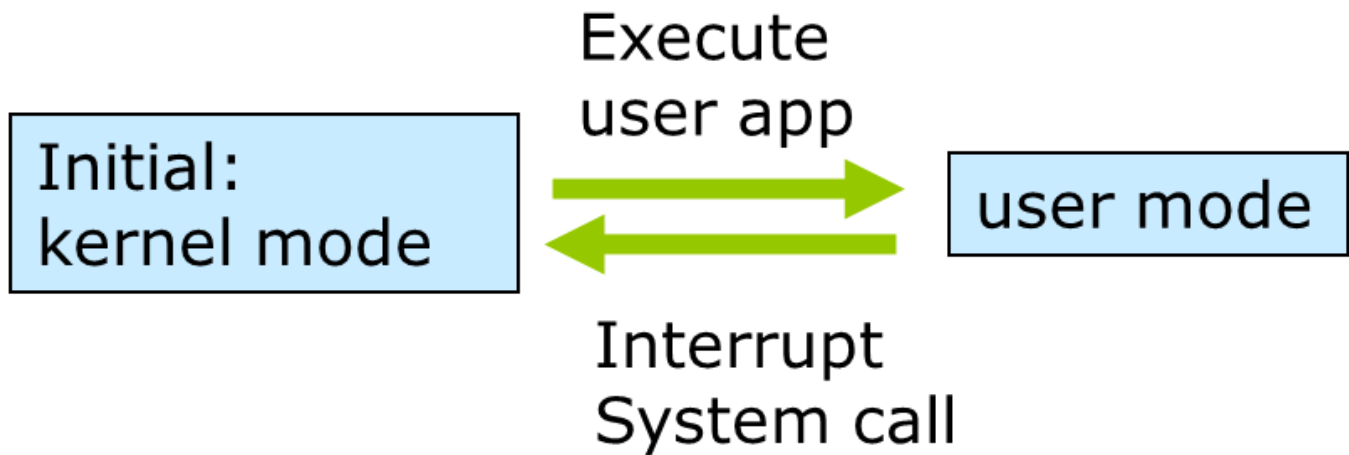
For operating-System Operations:

- User mode:
 - Each application has its private virtual address space and cannot modify another application's data.
 - Each application runs in isolation.
 - Virtual address space of a user-mode application is also limited.
- Kernel mode:
 - All code shares a single virtual address space
 - If it exists wrong operation to the virtual address, it could compromise the operating system or another driver.
 - If a kernel-mode driver crashes, it may cause the entire operating system to crash.

Mode bit are provided by hardware to distinguish when system is running user or kernel code. System call changes mode to kernel and return to user after resetting.

Some instructions designated as privileged and only executable in kernel mode.

Life cycle:



System Call

- **System call** provides the services of the operating system to the user programs through **Application Program Interface(API)**.
- It provides an **interface** between a process and operating system to allow user-level processes to request services of the operating system.
- And it is **the only entry points into kernel system**.

Multiprogramming (Batch System)

Multiprogramming organizes jobs(code and data) so **CPU always has one to execute** since user cannot always keep CPU and I/O devices busy.

- Some jobs in system is kept in memory.
- One job selected and run via **job scheduling**.
- When job has to wait, OS **switches to another job**.

Multitasking (Timesharing)

Multitasking is **a logical extension of Batch system(Multiprogramming)**. The CPU switches jobs frequently so that users can interact with each job while it is running, creating **interactive computing**.

- **Response time** is less than 1 second
- Each user has at least **one program** executing in memory.(called **process**)
- If several jobs ready to run at the same time, it needs **CPU scheduling**.
- If processes don't fit in memory, **swapping** moves them in and out to run.
- **Virtual memory** allows execution of processes not completely in memory (might extension in disks)

Timer

Timer is to prevent infinite loop(or process hogging resources)

- Interrupt the computer after some time period
- Keep a counter that is decremented by the physical clock. The counter is set by OS in privileged instruction.
- Generate an interrupt when counter zero.
- Set up before scheduling process to regain control or terminate program that exceeds allotted time.

Process Management

What is Process

A process is a program in execution as a unit of work within the system. (Program is passive entity and process is an active entity)

- Process needs resources to accomplish the task(CPU, memory, I/O files)
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one program counter specifying location of next instruction to execute. And process executes them sequentially and one at a time until accomplishment. Multi-thread process has one program counter per thread.
- System can have many processes concurrently by multiplexing the CPUs among processes/threads

Process Management Activities

The OS is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization, process communication and deadlock handling.

Memory Management

- To execute a program all(or part) of the instructions must be in memory.
- All(or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when optimizing CPU utilization and computer response to users.
- Memory management activities:

- Keep track of which parts of memory are currently being used and by whom
- Deciding which processes and data(or parts of them) to move into or out of memory
- Allocating and deallocating memory space as needed.

File-system Management

OS provides uniform, logical view of information storage by abstracting physical properties to logical storage unit--file.

File usually organized into directories. And OS accesses control on most systems to determine who can access what files.

File-System management activities:

- Creating and deleting files and directories
- Primitives to manipulate files and directories
- Mapping files onto secondary storage
- Backup files onto stable(non-volatile) storage media

Mass-Storage Management

Usually disks used to store data that does not fit in main memory or data that must be kept for a long period of time. Entire speed of computer operation hinges on disk subsystem and its algorithm.

Mass-storage management activities:

- Mounting and unmounting(挂载与卸载)
- Free-space management
- Storage allocation
- Disk scheduling
- Partitioning
- Protection

Caching

Caching is an important principle performed at many levels in a computer(hardware, OS, software), copying the information in use from slower to faster storage temporarily.

When operation manipulates data, faster storage(cache) checked first to determine if information is there. If it is, directly from the cache; if not, copy to cache and use it.(Hierarchy)

Cache is **smaller than storage being cached**. Thus cache management, cache size and replacement policy is important.

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

And movement between levels can be explicit or implicit.

Migration of data "A" from Disk to Register

- Multitasking environment
Use **most recent value** no matter where it is stored in the storage hierarchy
- Multiprocessor environment
Must provide **cache coherency** in hardware such that **all CPUs have the most recent value in their cache**.
- Distributed environment
Several copies of a datum can exist.

I/O Subsystem

One purpose of OS is to **hide peculiarities of hardware devices from users**.

I/O subsystem responsible for

- Memory management of I/O including buffer(storing data temporarily while it is transferred), caching(storing parts of data in faster storage for performance), spooling(the overlapping of output of one job with input of other jobs)
- General device-driver interface
- Drivers for specific hardware devices

Protection and Security

- Protection:
Any **mechanism** for **controlling access of process or users** defined by OS

- Security:
Defense of the system against internal and external attacks (like worms, viruses, etc.)

Systems generally first distinguish among users, to determine who can do what

- User identities (User IDs) include name and associated number and one per user. It is associated with all files, processes of that user to determine access control
- Group identifier (Group ID) allows set of users to be defined and controls managed and also associated with files and processes.
- Privilege escalation allows user to change to effective ID with more rights

Virtualization

Virtualization allows operating system to run applications within other OSes and is a vast and growing industry.

- Emulation
Used when source CPU type different from target type. Generally slowest method. When computer language not compiled to native code--interpretation
- Virtualization
OS natively compiled for CPU running guest OSes also natively compiled (same type).
Virtual machine manager (VMM) provides virtualization services

Distributed Systems

Distributed systems are collection of separate, possibly heterogeneous, systems networked together.

Network is a communications path, and TCP/IP is most common protocols. The types of Network:

- Local area network (LAN)
- Wide area network (WAN)
- Metropolitan area network (MAN)
- Personal Area network (PAN)

Network Operating System provides features between systems across network.

- Communication scheme allows systems to exchange messages
- Illusion of a single system

Computer-System Architecture

Most systems use a single general-purpose processor and special-purpose processors as well.

- Multiprocessors systems(parallel systems, tightly-coupled systems)
 - Advantages:
 - Increased throughput
 - Economy of scale
 - Increased reliability
 - Two types
 - Asymmetric multiprocessing
Each processor is assigned a specific task
 - Symmetric multiprocessing
Each processor performs all tasks

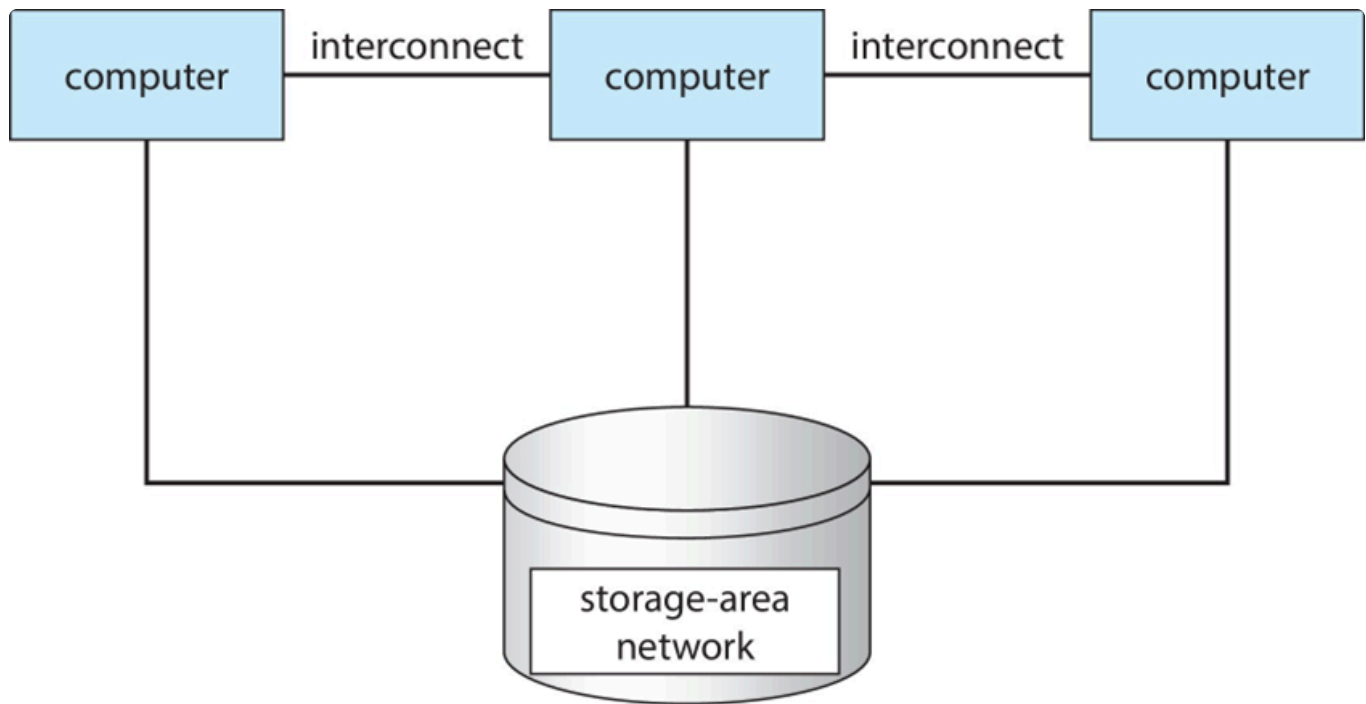
Dual-Core Design

Multi-chip and multicore, multiple separate systems

Clustered Systems

Multiple systems work together.

- Usually sharing storage via a storage-area network(SAN)
- Provides a high-availability service
- Two types
 - Asymmetric clustering
One machine in hot-standby mode(waiting for use forever)
 - Symmetric clustering
Multiple nodes running applications and monitoring each other
- High-performance computing(HPC)
Applications must be written to use parallelization
- Distributed lock manager(DLM): avoid conflicting operations



Computer System Environments

- Traditional
- Mobile
- Client Server
- Peer-to-Peer
- Cloud computing
- Real-time Embedded

Traditional

- **Stand-alone** general-purpose machines(blurred as most system interconnect with others such as Internet)
- **Portals** provides web access to internal systems
- **Network computers(thin clients)** are like Web terminals
- Use **firewalls** to protect from Internet attacks

Mobile

- Handheld smartphones, tablets, etc.
- Allows new type of apps
- Use IEEE 802.11 wireless or cellular data networks for connectivity

Client Server

- Client-Server Computing

Many systems now **servers**, responding to requests generated by **clients**.

- Compute-server system: provides an interface to client to request services
- File-server system: provides interface for client to store and retrieve files

P2P

- **Does not distinguish clients and servers**
 - All nodes are considered peers
 - Nodes must join P2P network, registering its service with central lookup service on network or broadcasting request for service and responding to requests for service via discovery protocol

Cloud Computing

- Delivers computing, storage even apps as a service across a network.
- Logical extension of **virtualization**
- Composed of traditional Oses, plus VMMs, cloud management tools(like firewalls, load balancers)

Many types:

- Public cloud: available via Internet to anyone willing to pay
- Private cloud: run for private company
- Hybrid cloud: both
- Software as a Service(SaaS): one or more applications available via the Internet
- Platform as a Service(PaaS): software stack ready for application use via the Internet
- Infrastructure as a Service(IaaS): servers or storage available via the Internet

Real-Time Embedded Systems

- Most prevalent form of computers, varying considerable.
- Well-defined fixed time constraints
 - **Must** be done within constraint
 - Correct operations only if constraints met

Free and Open-Source Operating Systems

- Available in **source-code format** rather than binary closed-source and proprietary, counter to the copy protection and Digital Rights management(DRM)
- Like GNU/Linux and BSD/Unix